

Computer Organization and Architecture

By: Karima Asaad

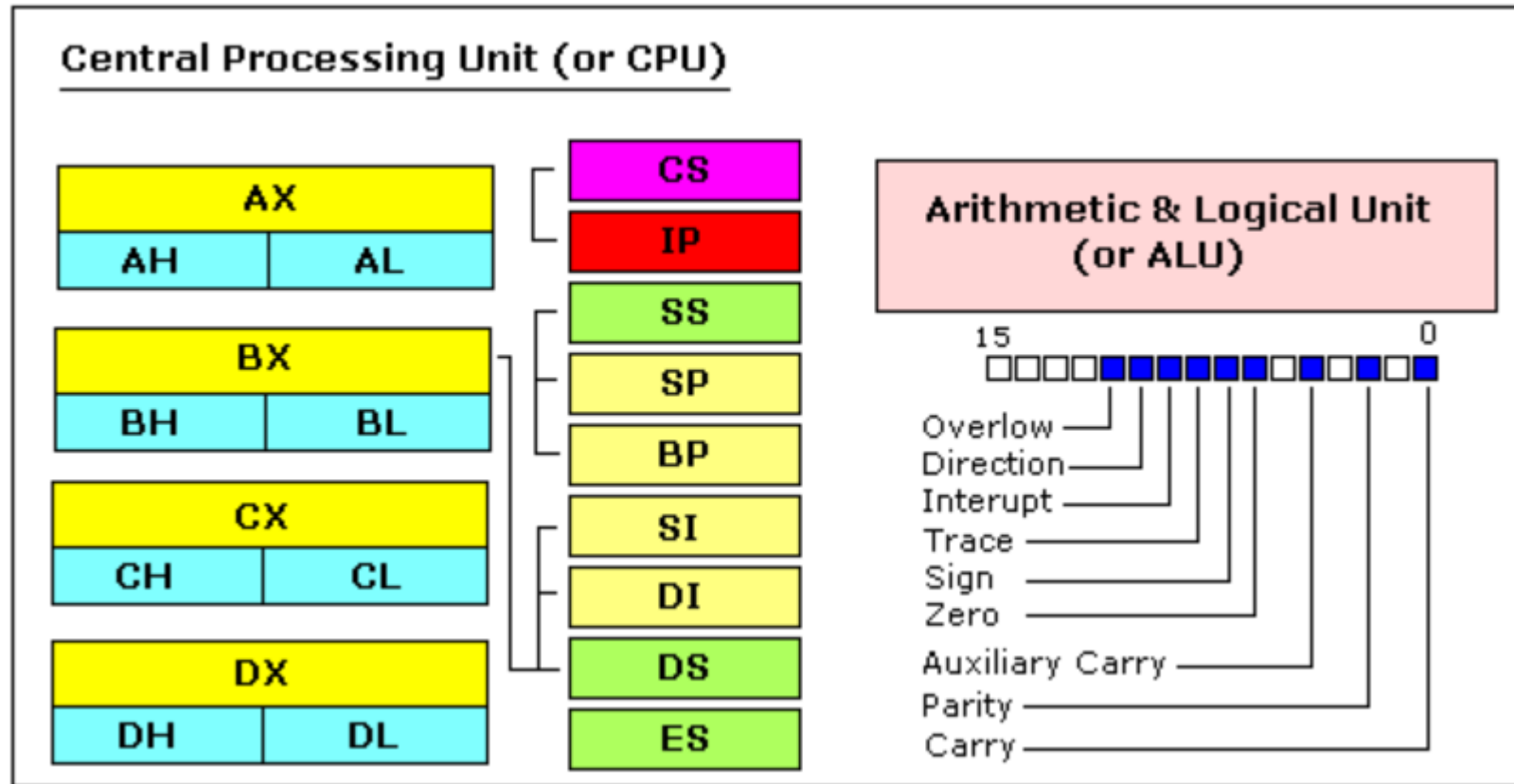
Agenda

- Assembly Language
- Variables
- Constants
- Arithmetic Instruction
- Macros & Procedures
- Arrays
- Loop
- Conditional & Unconditional Jump

What is Assembly Language?

- Each personal computer has a microprocessor that manages the computer's arithmetical, logical, and control activities.
- Each family of processors has its own set of instructions for handling various operations. these set of instructions are called 'machine language instructions'.
- So, the low-level assembly language is designed for a specific family of processors that represents various instructions in symbolic code and a more understandable form.

Central Processing Unit



General Purpose Registers

- **AX** - the accumulator register (divided into **AH** / **AL**).
- **BX** - the base address register (divided into **BH** / **BL**).
- **CX** - the count register (divided into **CH** / **CL**).
- **DX** - the data register (divided into **DH** / **DL**).
- **SI** - source index register.
- **DI** - destination index register.
- **BP** - base pointer.
- **SP** - stack pointer.

Segment Registers

- **CS** - points at the segment containing the current program.
 - **DS** - generally points at segment where variables are defined.
 - **ES** - extra segment register, it's up to a coder to define its usage.
 - **SS** - points at the segment containing the stack.
- the segment registers have a very special purpose - pointing at accessible blocks of memory.

Special Purpose Registers

- **IP** - the instruction pointer.
- **flags register** - determines the current state of the microprocessor.
- **IP** register always works together with **CS** segment register and it points to currently executing instruction.
- **flags register** is modified automatically by CPU after mathematical operations, this allows to determine the type of the result, and to determine conditions to transfer control to other parts of the program.

Addressing Mode Of Memory

- To access memory we can use these four registers: **BX, SI, DI, BP**.

$[BX + SI]$ $[BX + DI]$ $[BP + SI]$ $[BP + DI]$	$[SI]$ $[DI]$ d16 (variable offset only) $[BX]$	$[BX + SI + d8]$ $[BX + DI + d8]$ $[BP + SI + d8]$ $[BP + DI + d8]$
$[SI + d8]$ $[DI + d8]$ $[BP + d8]$ $[BX + d8]$	$[BX + SI + d16]$ $[BX + DI + d16]$ $[BP + SI + d16]$ $[BP + DI + d16]$	$[SI + d16]$ $[DI + d16]$ $[BP + d16]$ $[BX + d16]$

Example

- Assume that **DS = 100, BX = 30, SI = 70**.
The following addressing mode: **[BX + SI] + 25**
is calculated by processor .
- physical address:
- **$100 * 16 + 30 + 70 + 25 = 1725$**

Examples

- CPU makes a calculation of physical address by multiplying the segment register by 10h and adding general purpose register to it.
- **Example 1:**
 - The address formed with 2 registers **DS = 1230h** and **SI = 0045h**
 - physical address
 - $(1230h * 10h + 45h) = 12345h$
- **Example 2:**
 - When **DS** contains value **1234h** and **SI** contains the value **7890h** it can be also recorded as 1234:7890.
 - The physical address
$$1234h * 10h + 7890h = 19BD0h$$

Examples

- `org 100h`
- `mov ax,1234h`
- `mov ds,ax`
- `mov cx,'A'`
- `mov bx,15h`
- `mov [bx],cx`
- `ret`

Variables

- Variable is a memory location.
- Syntax for a variable declaration:

name **DB** value

name **DW** value

DB - stays for Define Byte.

DW - stays for Define Word.

name - can be any letter or digit combination, though it should start with a letter.

value - can be any numeric value in any supported numbering system (hexadecimal, binary, or decimal), or "?" symbol for variables that are not initialized.

Example

- ORG 100h
- MOV AL, var1
- MOV BX, var2
- RET ; stops the program.
- VAR1 DB 7
- var2 DW 1234h

Constants

- Constants are just like variables. After definition of a constant its value cannot be changed.
- To define constants **EQU** directive is used:
name **EQU** < any expression >
- For example:
k EQU 5
MOV AX, k
- The above example is functionally identical to code:
- MOV AX, 5

Library of common functions: emu8086.inc

- **emu8086.inc** defines the following macros:
 - **PRINT string** - macro with 1 parameter, prints out a string.
 - **PRINTN string** - macro with 1 parameter, prints out a string. The same as PRINT but automatically adds "carriage return" at the end of the string.
 - **PUTC char** - macro with 1 parameter, prints out an ASCII char at current cursor position.
 - **GOTOXY col, row** - macro with 2 parameters, sets cursor position.
 - **CURSOROFF** - turns off the text cursor.
 - **CURSORON** - turns on the text cursor.

Procedures

- **emu8086.inc** also defines the following procedures:

PRINT_NUM - procedure that prints a signed number in **AX** register. To use it declare: **DEFINE_PRINT_NUM** and **DEFINE_PRINT_NUM_UN** before **END** directive.

- **PRINT_NUM_UN** - procedure that prints out an unsigned number in **AX** register. To use it declare: **DEFINE_PRINT_NUM_UN** before **END** directive.

- **GET_STRING** - procedure to get a null terminated string from a user, the received string is written to buffer at **DS:DI**, buffer size should be in **DX**. Procedure stops the input when 'Enter' is pressed. To use it declare: **DEFINE_GET_STRING** before **END** directive.

- **PRINT_STRING** - procedure to print a null terminated string at current cursor position, receives address of string in **DS:SI** register. To use it declare: **DEFINE_PRINT_STRING** before **END** directive.

procedures

- **CLEAR_SCREEN** - procedure to clear the screen, (done by scrolling entire screen window), and set cursor position to top of it. To use it declare: **DEFINE_CLEAR_SCREEN** before **END** directive.
- **SCAN_NUM** - procedure that gets the multi-digit SIGNED number from the keyboard, and stores the result in **CX** register. To use it declare: **DEFINE_SCAN_NUM** before **END** directive.

Array

- Arrays can be seen as chains of variables. A text string is an example of a byte array, each character is presented as an ASCII code value (0..255).
- Here are some array definition examples:
 - a DB 48h, 65h, 6Ch, 6Ch, 6Fh, 00h
 - b DB 'Hello', 0

Array

- If you need to declare a large array you can use **DUP** operator.
The syntax for **DUP**:

number DUP (value(s))

number - number of duplicate to make (any constant value).

value - expression that DUP will duplicate.

for example:

c DB 5 DUP(9)

is an alternative way of declaring:

c DB 9, 9, 9, 9, 9

one more example:

d DB 5 DUP(1, 2)

is an alternative way of declaring:

d DB 1, 2, 1, 2, 1, 2, 1, 2, 1, 2

unconditional jumps

- The basic syntax of **JMP** instruction:
- JMP label
- label: